

DEPSTAR
Department of Computer Science & Engineering
CSPIT, CHARUSAT

Exception Handling

in Java

Comprehensive Class Notes

Topics Covered:

- Introduction to Exception Handling
- Exception Hierarchy
- Checked vs Unchecked Exceptions
- Try-Catch-Finally Blocks
- Throw and Throws Keywords
- Custom Exceptions
- Best Practices and Use Cases

Academic Year: 2026-267

Course: Object-Oriented Programming

Prepared by: Mr. Ronak Sir and Team

August 19, 2026

Contents

1	Introduction to Exception Handling	2
1.1	Why Exception Handling?	2
1.2	CSPIT-CSE Real-World Analogy	2
2	Exception Hierarchy in Java	2
2.1	Complete Exception Hierarchy	2
2.2	Types of Exceptions	3
2.3	CSPIT Example: Understanding Exception Types	3
3	Exception Handling Keywords	5
3.1	The Five Keywords	5
3.2	Try-Catch Block	5
3.3	Multiple Catch Blocks	6
3.4	The Finally Block	8
4	Throw and Throws	9
4.1	The Throw Keyword	9
4.2	The Throws Keyword	11
5	Custom Exceptions	13
5.1	Creating Custom Exceptions	13
6	Best Practices and Guidelines	16
6.1	CSPIT-CSE Exception Handling Guidelines	17
6.2	Common Mistakes to Avoid	19
7	Real-World CSPIT-CSE Examples	20
7.1	Complete Student Management System	20
8	Exception Handling Summary	28
8.1	Key Takeaways	28
8.2	CSPIT Implementation Checklist	28

1 Introduction to Exception Handling

Definition

An **exception** is an unwanted or unexpected event that occurs during the execution of a program, disrupting the normal flow of instructions. **Exception Handling** is the mechanism to handle runtime errors and maintain the normal flow of the application.

1.1 Why Exception Handling?

- **Program Robustness:** Prevents abrupt termination of programs
- **Error Reporting:** Provides meaningful error messages
- **Separation of Concerns:** Separates error handling from business logic
- **Resource Management:** Ensures proper cleanup of resources
- **Debugging:** Helps identify and fix issues efficiently

1.2 CSPIT-CSE Real-World Analogy

Important

Think of exception handling like the **CSPIT Student Registration System**:

- **Normal Flow:** Student fills form → System validates → Registration successful
- **Exception:** Student enters invalid ID → System throws validation error → User corrects input
- **Handling:** The system gracefully handles the error without crashing

2 Exception Hierarchy in Java

2.1 Complete Exception Hierarchy

```
1 // The root class of all exceptions
2 java.lang.Object
3 |
4 +-- java.lang.Throwable
5 |
6 +-- java.lang.Exception (Checked Exceptions)
7 |   |
8 |   +-- java.lang.ClassNotFoundException
9 |   +-- java.io.IOException
10 |   |   |
11 |   |   +-- java.io.FileNotFoundException
12 |   |   +-- java.io.EOFException
13 |   +-- java.sql.SQLException
14 |   +-- java.lang.InterruptedException
```

```

15         |         +-- java.lang.CloneNotSupportedException
16         |
17         +-- java.lang.Error (Unchecked - Serious Issues)
18         |
19         +-- java.lang.OutOfMemoryError
20         +-- java.lang.StackOverflowError
21         +-- java.lang.VirtualMachineError
22         +-- java.lang.AssertionError
23
24 // RuntimeException hierarchy (Unchecked Exceptions)
25 java.lang.RuntimeException
26     |
27     +-- java.lang.ArithmeticException
28     +-- java.lang.ArrayIndexOutOfBoundsException
29     +-- java.lang.NullPointerException
30     +-- java.lang.NumberFormatException
31     +-- java.lang.IllegalArgumentException
32     +-- java.lang.ClassCastException
33     +-- java.lang.StringIndexOutOfBoundsException
34     +-- java.lang.NegativeArraySizeException

```

Listing 1: Exception Hierarchy (Based on Java Complete Reference)

2.2 Types of Exceptions

Type	Category	Description
Checked Exceptions	Exception (subclass)	Must be handled at compile time. The compiler forces you to either catch or declare them.
Unchecked Exceptions	RuntimeException	Occur at runtime. Compiler doesn't enforce handling. Usually due to programming errors.
Errors	Error	Serious issues beyond the program's control. Usually not handled by applications.

Table 1: Exception Categories

2.3 CSPIT Example: Understanding Exception Types

```

1 import java.io.*;
2 import java.util.*;
3
4 public class CSPITExceptionDemo {
5
6     // 1. Checked Exception Example
7     public static void readStudentData() throws IOException {
8         // This is a checked exception - must be handled or
9         // declared
10    }
11 }

```

```
9      FileReader file = new FileReader("students.txt");
10     BufferedReader reader = new BufferedReader(file);
11     String studentData = reader.readLine();
12     System.out.println("Student Data: " + studentData);
13     reader.close();
14 }
15
16 // 2. Unchecked Exception Example
17 public static void processStudentMarks(int[] marks) {
18     // NullPointerException - if marks array is null
19     // ArrayIndexOutOfBoundsException - if index is invalid
20     // ArithmeticException - if division by zero
21     for (int i = 0; i < marks.length; i++) {
22         int percentage = (marks[i] / 10) * 10; // Could cause
23         // division by zero
24         System.out.println("Student " + i + " Percentage: " +
25             percentage);
26     }
27 }
28
29 // 3. Error Example
30 public static void recursiveMethod() {
31     // StackOverflowError - unbounded recursion
32     recursiveMethod();
33 }
34
35 public static void main(String[] args) {
36     System.out.println("=== CSBIT Exception Types
37     Demonstration ===\n");
38
39     // Demonstrating Unchecked Exception
40     try {
41         int[] marks = {85, 92, 78, 95};
42         processStudentMarks(marks);
43
44         // This will cause ArrayIndexOutOfBoundsException
45         System.out.println(marks[10]);
46     } catch (ArrayIndexOutOfBoundsException e) {
47         System.out.println("Error: Attempted to access
48         invalid index!");
49     } catch (NullPointerException e) {
50         System.out.println("Error: Marks array is null!");
51     }
52
53     // Demonstrating Checked Exception
54     try {
55         readStudentData();
56     } catch (IOException e) {
57         System.out.println("Error reading student data: " + e
58             .getMessage());
59     }
```

```
55         System.out.println("\nProgram continues normally...");
56     }
57 }
58 }
```

Listing 2: Different Types of Exceptions in CSPIT System

3 Exception Handling Keywords

3.1 The Five Keywords

Java provides five keywords for exception handling:

- **try**: Block of code that might throw an exception
- **catch**: Block that handles the exception
- **finally**: Block that executes regardless of exception occurrence
- **throw**: Used to explicitly throw an exception
- **throws**: Declares that a method might throw exceptions

3.2 Try-Catch Block

```
1  // Basic syntax
2  try {
3      // Code that might throw an exception
4      // Risky operations go here
5  } catch (ExceptionType e) {
6      // Code to handle the exception
7      // Log the error, display message, recover from error
8  } catch (AnotherExceptionType e) {
9      // Multiple catch blocks can be used
10 } finally {
11     // Code that always executes (optional)
12     // Cleanup operations like closing files, database
13     // connections
14 }
15
16 // CSPIT Example: Student Registration System
17 class StudentRegistration {
18     private String studentId;
19     private String name;
20     private int age;
21
22     public void registerStudent(String studentId, String name,
23     int age) {
24         this.studentId = studentId;
25         this.name = name;
26         this.age = age;
27     }
28 }
```

```
25
26     try {
27         // Validate student ID format
28         if (!studentId.matches("CSPIT\\d{7}")) {
29             throw new IllegalArgumentException("Invalid
30                 Student ID format! Must be 'CSPIT' followed by
31                 7 digits.");
32         }
33
34         // Validate age
35         if (age < 18 || age > 60) {
36             throw new IllegalArgumentException("Invalid age!
37                 Must be between 18 and 60.");
38         }
39
40         // Simulate database operation
41         System.out.println("Registering student: " + name + "
42             (" + studentId + ")");
43         System.out.println("Registration successful!");
44
45     } catch (IllegalArgumentException e) {
46         System.out.println("Validation Error: " + e.
47             getMessage());
48     } catch (Exception e) {
49         System.out.println("Unexpected Error: " + e.
50             getMessage());
51     } finally {
52         System.out.println("Registration process completed.")
53         ;
54     }
55 }
56
57 public class RegistrationDemo {
58     public static void main(String[] args) {
59         StudentRegistration reg = new StudentRegistration();
60
61         System.out.println("=== CSPIT Student Registration ===\n"
62             );
63         reg.registerStudent("CSPIT2025001", "Rahul Sharma", 20);
64         System.out.println();
65         reg.registerStudent("CSPIT2025", "Priya Patel", 22);
66         System.out.println();
67         reg.registerStudent("CSPIT2025002", "Amit Kumar", 17);
68     }
69 }
```

Listing 3: Try-Catch Syntax and Example

3.3 Multiple Catch Blocks

```
1 public class CSPITMultiCatchDemo {
2
3     public static void processStudentData(String data) {
4         try {
5             // Parse integer from string
6             int studentId = Integer.parseInt(data);
7
8             // Division operation
9             int result = 100 / studentId;
10
11             // Array operations
12             String[] studentNames = {"Rahul", "Priya", "Amit"};
13             System.out.println("Student at index " + studentId +
14                 ": " + studentNames[studentId]);
15
16             } catch (NumberFormatException e) {
17                 System.out.println("NumberFormatException: Invalid
18                     number format - " + e.getMessage());
19             } catch (ArithmeticException e) {
20                 System.out.println("ArithmeticException: Cannot
21                     divide by zero!");
22             } catch (ArrayIndexOutOfBoundsException e) {
23                 System.out.println("ArrayIndexOutOfBoundsException:
24                     Student ID is out of range!");
25             } catch (Exception e) {
26                 System.out.println("Generic Exception: " + e.
27                     getMessage());
28             }
29         }
30
31     public static void main(String[] args) {
32         System.out.println("=== CSPIT Multi-Catch Demonstration
33             ===\n");
34
35         // Test with valid data
36         System.out.println("Test 1 - Valid Data:");
37         processStudentData("2");
38
39         // Test with non-numeric data
40         System.out.println("\nTest 2 - Non-numeric Data:");
41         processStudentData("CSPIT");
42
43         // Test with zero
44         System.out.println("\nTest 3 - Division by Zero:");
45         processStudentData("0");
46
47         // Test with large index
48         System.out.println("\nTest 4 - Index Out of Bounds:");
49         processStudentData("10");
50     }
```



```
}
```

Listing 4: Multiple Catch Blocks Example

3.4 The Finally Block

```
1 import java.io.*;
2
3 public class CSPITFinallyDemo {
4
5     // Proper resource management using finally
6     public static String readStudentFile(String filename) {
7         BufferedReader reader = null;
8         String content = "";
9
10        try {
11            reader = new BufferedReader(new FileReader(filename))
12                ;
13            String line;
14            while ((line = reader.readLine()) != null) {
15                content += line + "\n";
16            }
17            return content;
18        } catch (FileNotFoundException e) {
19            System.out.println("Error: File not found - " +
20                filename);
21            return null;
22        } catch (IOException e) {
23            System.out.println("Error: IOException occurred - " +
24                e.getMessage());
25            return null;
26        } finally {
27            // Cleanup code - always executes
28            try {
29                if (reader != null) {
30                    reader.close();
31                    System.out.println("File closed successfully.
32                        ");
33                }
34            } catch (IOException e) {
35                System.out.println("Error closing file: " + e.
36                    getMessage());
37            }
38        }
39    }
40
41    // Java 7+ try-with-resources (better approach)
42    public static String readStudentFileModern(String filename) {
43        StringBuilder content = new StringBuilder();
```

```
41     try (BufferedReader reader = new BufferedReader(new
42           FileReader(filename))) {
43         String line;
44         while ((line = reader.readLine()) != null) {
45             content.append(line).append("\n");
46         }
47         return content.toString();
48     } catch (FileNotFoundException e) {
49         System.out.println("Error: File not found - " +
50           filename);
51         return null;
52     } catch (IOException e) {
53         System.out.println("Error: IOException occurred - " +
54           e.getMessage());
55         return null;
56     }
57     // File automatically closed by try-with-resources
58 }
59
60 public static void main(String[] args) {
61     System.out.println("=== CSPIT File Reading Demonstration
62       ===\n");
63
64     String filename = "student_data.txt";
65     System.out.println("Reading file: " + filename);
66
67     String data = readStudentFile(filename);
68     if (data != null) {
69         System.out.println("File content:\n" + data);
70     }
71 }
```

Listing 5: Finally Block - Resource Management

4 Throw and Throws

4.1 The Throw Keyword

throw is used to explicitly throw an exception from a method or block.

```
1 class CSPITStudentValidator {
2
3     // Method using throw to validate student data
4     public static void validateStudent(String studentId, int
5       marks)
6         throws IllegalArgumentException {
7
8         if (studentId == null || studentId.trim().isEmpty()) {
9             throw new IllegalArgumentException("Student ID cannot
10              be null or empty");
11         }
12     }
13 }
```

```
9      }
10
11      if (!studentId.matches("CSPIT\\d{7}")) {
12          throw new IllegalArgumentException("Student ID must
13              start with 'CSPIT' and contain 7 digits");
14      }
15
16      if (marks < 0 || marks > 100) {
17          throw new IllegalArgumentException("Marks must be
18              between 0 and 100");
19      }
20
21      System.out.println("Student validation passed!");
22  }
23
24  public static void main(String[] args) {
25      System.out.println("=== CSPIT Student Validation ===\n");
26
27      try {
28          System.out.println("Test 1 - Valid Student:");
29          validateStudent("CSPIT2025001", 85);
30
31          System.out.println("\nTest 2 - Invalid Student ID:");
32          validateStudent("CSPIT2025", 85);
33
34      } catch (IllegalArgumentException e) {
35          System.out.println("Validation Error: " + e.
36              getMessage());
37      }
38
39      try {
40          System.out.println("\nTest 3 - Invalid Marks:");
41          validateStudent("CSPIT2025002", 105);
42
43      } catch (IllegalArgumentException e) {
44          System.out.println("Validation Error: " + e.
45              getMessage());
46      }
47
48      // Custom exception usage
49      try {
50          System.out.println("\nTest 4 - Using Custom Exception
51              :");
52          checkScholarshipEligibility("CSPIT2025003", 6.5);
53      } catch (ScholarshipException e) {
54          System.out.println("Scholarship Error: " + e.
55              getMessage());
56      }
57  }
58
59  // Example with custom exception
```

```
54     public static void checkScholarshipEligibility(String
        studentId, double cgpa)
55         throws ScholarshipException {
56
57         if (cgpa < 8.0) {
58             throw new ScholarshipException("CGPA " + cgpa + " is
                below 8.0. Not eligible for scholarship.");
59         }
60         System.out.println("Student " + studentId + " is eligible
                for scholarship!");
61     }
62 }
63
64 // Custom Exception Class
65 class ScholarshipException extends Exception {
66     public ScholarshipException(String message) {
67         super(message);
68     }
69 }
```

Listing 6: Using Throw Keyword

4.2 The Throws Keyword

throws is used in method declaration to indicate that the method might throw exceptions.

```
1  import java.io.*;
2
3  class CSPITDataProcessor {
4
5      // Method declaring checked exceptions using throws
6      public void processStudentRecord(String filename, int[] marks
        )
7          throws FileNotFoundException, IOException,
            ArithmeticException {
8
9          // This might throw FileNotFoundException if file doesn't
            exist
10         BufferedReader reader = new BufferedReader(new FileReader
            (filename));
11
12         // This might throw IOException if reading fails
13         String line = reader.readLine();
14         System.out.println("Data: " + line);
15
16         // This might throw ArithmeticException if division by
            zero
17         int average = calculateAverage(marks);
18         System.out.println("Average: " + average);
19
20         reader.close();
```

```
21     }
22
23     private int calculateAverage(int[] marks) {
24         int sum = 0;
25         for (int mark : marks) {
26             sum += mark;
27         }
28         return sum / marks.length; // Potential division by zero
29     }
30
31     // Propagating exceptions to caller
32     public void processBulkRecords(String[] files) throws
33     IOException {
34         for (String file : files) {
35             try {
36                 processStudentRecord(file, new int[]{85, 92, 78})
37                 ;
38             } catch (FileNotFoundException e) {
39                 System.out.println("Skipping missing file: " +
40                     file);
41             } catch (ArithmeticException e) {
42                 System.out.println("Skipping invalid data in: " +
43                     file);
44             }
45         }
46     }
47 }
48
49 public class ThrowsDemo {
50     public static void main(String[] args) {
51         System.out.println("=== CSPIT Throws Keyword
52         Demonstration ===\n");
53
54         CSPITDataProcessor processor = new CSPITDataProcessor();
55
56         try {
57             // This might throw exceptions, handled by caller
58             processor.processStudentRecord("students.txt", new
59             int[]{85, 92, 78});
60
61         } catch (FileNotFoundException e) {
62             System.out.println("File not found error: " + e.
63                 getMessage());
64         } catch (IOException e) {
65             System.out.println("I/O error: " + e.getMessage());
66         } catch (ArithmeticException e) {
67             System.out.println("Arithmetic error: " + e.
68                 getMessage());
69         }
70
71         // Processing multiple files
```

```
64     String[] files = {"student1.txt", "student2.txt", "
        student3.txt"};
65     try {
66         processor.processBulkRecords(files);
67     } catch (IOException e) {
68         System.out.println("Error in bulk processing: " + e.
            getMessage());
69     }
70 }
71 }
```

Listing 7: Using Throws Keyword

5 Custom Exceptions

5.1 Creating Custom Exceptions

Custom exceptions are useful for application-specific error conditions.

```
1 // Custom Checked Exception
2 class CSPITAcademicException extends Exception {
3
4     private String errorCode;
5     private String studentId;
6
7     public CSPITAcademicException(String message) {
8         super(message);
9     }
10
11     public CSPITAcademicException(String message, String
        errorCode) {
12         super(message);
13         this.errorCode = errorCode;
14     }
15
16     public CSPITAcademicException(String message, String
        errorCode, String studentId) {
17         super(message);
18         this.errorCode = errorCode;
19         this.studentId = studentId;
20     }
21
22     // Getters for additional info
23     public String getErrorCode() { return errorCode; }
24     public String getStudentId() { return studentId; }
25
26     @Override
27     public String toString() {
28         return "CSPITAcademicException [ErrorCode: " + errorCode
            +
29             ", StudentId: " + studentId + ", Message: " +
                getMessage() + " ]";
    }
```

```
30     }
31 }
32
33 // Custom Unchecked Exception
34 class CSPITSystemException extends RuntimeException {
35
36     public CSPITSystemException(String message) {
37         super(message);
38     }
39
40     public CSPITSystemException(String message, Throwable cause)
41     {
42         super(message, cause);
43     }
44 }
45
46 // Example usage of custom exceptions
47 class CSPITStudentManager {
48
49     public void verifyStudentForGraduation(String studentId, int
50 credits, double cgpa)
51     throws CSPITAcademicException {
52
53         // Validate credits
54         if (credits < 180) {
55             throw new CSPITAcademicException(
56                 "Student hasn't completed required credits",
57                 "CREDIT_FAIL",
58                 studentId
59             );
60         }
61
62         // Validate CGPA
63         if (cgpa < 5.0) {
64             throw new CSPITAcademicException(
65                 "Student CGPA is below graduation requirement",
66                 "CGPA_FAIL",
67                 studentId
68             );
69         }
70
71         System.out.println("Student " + studentId + " is eligible
72 for graduation!");
73     }
74
75     public void processStudentData(String data) {
76         try {
77             int number = Integer.parseInt(data);
78             int result = 100 / number;
79             System.out.println("Result: " + result);
80         }
81     }
82 }
```

```
78     } catch (NumberFormatException e) {
79         throw new CSPITSystemException("Invalid number format
80         : " + data, e);
81     } catch (ArithmeticException e) {
82         throw new CSPITSystemException("Division by zero
83         error", e);
84     }
85 }
86
87 public class CustomExceptionDemo {
88     public static void main(String[] args) {
89         System.out.println("=== CSPIT Custom Exception
90         Demonstration ===\n");
91
92         CSPITStudentManager manager = new CSPITStudentManager();
93
94         // Test 1 - Student eligible
95         try {
96             System.out.println("Test 1: Eligible Student");
97             manager.verifyStudentForGraduation("CSPIT2025001",
98             190, 7.5);
99         } catch (CSPITAcademicException e) {
100             System.out.println("Error: " + e);
101         }
102
103         // Test 2 - Credits insufficient
104         try {
105             System.out.println("\nTest 2: Insufficient Credits");
106             manager.verifyStudentForGraduation("CSPIT2025002",
107             170, 7.5);
108         } catch (CSPITAcademicException e) {
109             System.out.println("Error: " + e);
110         }
111
112         // Test 3 - CGPA insufficient
113         try {
114             System.out.println("\nTest 3: Insufficient CGPA");
115             manager.verifyStudentForGraduation("CSPIT2025003",
116             190, 4.5);
117         } catch (CSPITAcademicException e) {
118             System.out.println("Error: " + e);
119         }
120
121         // Test 4 - System Exception
122         try {
123             System.out.println("\nTest 4: System Exception");
124             manager.processStudentData("CSPIT");
125         } catch (CSPITSystemException e) {
126             System.out.println("System Error: " + e.getMessage())
127             ;
128         }
129     }
130 }
```



```
122         System.out.println("Cause: " + e.getCause().  
123             getMessage());  
124     }  
125 }
```

Listing 8: Custom Exception Creation

6 Best Practices and Guidelines

6.1 CSPIT-CSE Exception Handling Guidelines

Best Practice

Best Practices for Exception Handling

1. Catch Specific Exceptions First:

```
1 // Good Practice
2 try {
3     // code
4 } catch (FileNotFoundException e) {
5     // handle file not found
6 } catch (IOException e) {
7     // handle IO errors
8 } catch (Exception e) {
9     // handle all other exceptions
10 }
11
12 // Bad Practice - Catching generic exception first
13 try {
14     // code
15 } catch (Exception e) {
16     // This catches everything, making specific catches
17     // unreachable
18 }
```

2. Use Finally Block for Cleanup:

```
1 // Good Practice
2 Connection conn = null;
3 try {
4     conn = DriverManager.getConnection(url);
5     // Use connection
6 } finally {
7     if (conn != null) {
8         try { conn.close(); } catch (SQLException e) { /*
9             log */ }
10     }
11 }
12
13 // Better Practice - Try-with-resources (Java 7+)
14 try (Connection conn = DriverManager.getConnection(url)) {
15     // Use connection - automatically closed
16 }
```

3. Don't Ignore Exceptions:

```
1 // Bad Practice - Swallowing exceptions
2 try {
3     // risky code
4 } catch (Exception e) {
5     // do nothing - dangerous!
6 }
7
8 // Good Practice - Log and/or rethrow
9 try {
10     // risky code
11 }
```


6.2 Common Mistakes to Avoid

Common Mistake

Common Mistakes in Exception Handling:

1. Catching Too Broad Exceptions:

```
1 // Bad - catches everything including runtime exceptions
2 try {
3     // code
4 } catch (Exception e) {
5     // handle all errors
6 }
7
8 // Better - be specific
9 try {
10    // code
11 } catch (IOException e) {
12    // handle IO errors
13 } catch (SQLException e) {
14    // handle SQL errors
15 }
```

2. Swallowing Exceptions:

```
1 // Bad - ignoring the exception
2 try {
3     // code
4 } catch (IOException e) {
5     // do nothing
6 }
7
8 // Good - at least log it
9 try {
10    // code
11 } catch (IOException e) {
12    logger.error("IO Error occurred", e);
13 }
```

3. Using Exception for Flow Control:

```
1 // Bad - using exception for flow control
2 try {
3     int value = Integer.parseInt(input);
4     // process value
5 } catch (NumberFormatException e) {
6     // treat as default value
7 }
8
9 // Better - validate first
10 if (input.matches("\\d+")) {
11     int value = Integer.parseInt(input);
12     // process value
13 } else {
14     // handle invalid input
15 }
```

7 Real-World CSPIT-CSE Examples

7.1 Complete Student Management System

```
1 import java.io.*;
2 import java.util.*;
3
4 // Custom Exceptions
5 class StudentNotFoundException extends Exception {
6     public StudentNotFoundException(String message) {
7         super(message);
8     }
9 }
10
11 class InvalidStudentDataException extends Exception {
12     public InvalidStudentDataException(String message) {
13         super(message);
14     }
15 }
16
17 class DatabaseConnectionException extends RuntimeException {
18     public DatabaseConnectionException(String message) {
19         super(message);
20     }
21 }
22
23 // Student Class
24 class Student {
25     private String studentId;
26     private String name;
27     private String department;
28     private int[] marks;
29     private double cgpa;
30
31     public Student(String studentId, String name, String
32         department, int[] marks)
33         throws InvalidStudentDataException {
34
35         // Validate student ID
36         if (studentId == null || !studentId.matches("CSPIT\\d{7}")
37             )) {
38             throw new InvalidStudentDataException("Invalid
39                 Student ID format: " + studentId);
40         }
41
42         // Validate name
43         if (name == null || name.trim().isEmpty()) {
44             throw new InvalidStudentDataException("Student name
45                 cannot be empty");
46         }
47     }
48 }
```

```
44 // Validate department
45 if (department == null || department.trim().isEmpty()) {
46     throw new InvalidStudentDataException("Department
47         cannot be empty");
48 }
49
50 // Validate marks
51 if (marks == null || marks.length == 0) {
52     throw new InvalidStudentDataException("Marks array
53         cannot be null or empty");
54 }
55
56 for (int mark : marks) {
57     if (mark < 0 || mark > 100) {
58         throw new InvalidStudentDataException("Invalid
59             mark: " + mark);
60     }
61 }
62
63 this.studentId = studentId;
64 this.name = name;
65 this.department = department;
66 this.marks = marks.clone(); // Defensive copy
67 this.cgpa = calculateCGPA(marks);
68 }
69
70 private double calculateCGPA(int[] marks) {
71     try {
72         int sum = 0;
73         for (int mark : marks) {
74             sum += mark;
75         }
76         double average = sum / (double) marks.length;
77         return average / 10.0; // Convert to 10-point scale
78     } catch (ArithmeticException e) {
79         throw new DatabaseConnectionException("Error
80             calculating CGPA: " + e.getMessage());
81     }
82 }
83
84 public String getStudentId() { return studentId; }
85 public String getName() { return name; }
86 public String getDepartment() { return department; }
87 public double getCgpa() { return cgpa; }
88 public int[] getMarks() { return marks.clone(); }
89
90 @Override
91 public String toString() {
92     return String.format(
93         "Student[ID=%s, Name=%s, Dept=%s, CGPA=%.2f]",
```

```
91         studentId, name, department, cgpa
92     );
93 }
94 }
95
96 // Student Database Manager
97 class StudentDatabaseManager {
98     private List<Student> students = new ArrayList<>();
99     private String databaseFile = "students.dat";
100
101     // Save students to file
102     public void saveToFile() throws IOException {
103         if (students.isEmpty()) {
104             throw new IOException("No students to save");
105         }
106
107         try (ObjectOutputStream oos = new ObjectOutputStream(
108             new FileOutputStream(databaseFile))) {
109             oos.writeObject(students);
110             System.out.println("Saved " + students.size() + "
111                 students to file.");
112         } catch (IOException e) {
113             throw new IOException("Failed to save student data: "
114                 + e.getMessage(), e);
115         }
116     }
117
118     // Load students from file
119     @SuppressWarnings("unchecked")
120     public void loadFromFile() throws IOException,
121         ClassNotFoundException {
122         try (ObjectInputStream ois = new ObjectInputStream(
123             new FileInputStream(databaseFile))) {
124             students = (List<Student>) ois.readObject();
125             System.out.println("Loaded " + students.size() + "
126                 students from file.");
127         } catch (FileNotFoundException e) {
128             System.out.println("No existing database file found.
129                 Starting fresh.");
130             students = new ArrayList<>();
131         } catch (IOException | ClassNotFoundException e) {
132             throw new IOException("Failed to load student data: "
133                 + e.getMessage(), e);
134         }
135     }
136
137     // Add student
138     public void addStudent(Student student) throws
139         StudentNotFoundException {
140         if (student == null) {
```

```
134         throw new StudentNotFoundException("Student cannot be
135             null");
136     }
137     // Check for duplicate
138     for (Student s : students) {
139         if (s.getStudentId().equals(student.getStudentId()))
140             {
141                 throw new StudentNotFoundException(
142                     "Student with ID " + student.getStudentId() +
143                     " already exists"
144                 );
145             }
146     }
147     students.add(student);
148     System.out.println("Student added successfully: " +
149         student.getName());
150 }
151 // Find student by ID
152 public Student findStudent(String studentId) throws
153     StudentNotFoundException {
154     for (Student s : students) {
155         if (s.getStudentId().equals(studentId)) {
156             return s;
157         }
158     }
159     throw new StudentNotFoundException("Student with ID " +
160         studentId + " not found");
161 }
162 // Get all students
163 public List<Student> getAllStudents() {
164     return Collections.unmodifiableList(students);
165 }
166 // Get students by department
167 public List<Student> getStudentsByDepartment(String
168     department) {
169     List<Student> result = new ArrayList<>();
170     for (Student s : students) {
171         if (s.getDepartment().equalsIgnoreCase(department)) {
172             result.add(s);
173         }
174     }
175     return result;
176 }
177 // Get toppers
178 public List<Student> getTopStudents(int topCount) {
```



```
178         if (topCount <= 0) {
179             throw new IllegalArgumentException("Top count must be
                positive");
180         }
181
182         List<Student> sortedStudents = new ArrayList<>(students);
183         sortedStudents.sort((s1, s2) -> Double.compare(s2.getCgpa
            (), s1.getCgpa()));
184
185         return sortedStudents.subList(0, Math.min(topCount,
            sortedStudents.size()));
186     }
187 }
188
189 // Main Application
190 public class CSPITStudentManagementSystem {
191     private static StudentDatabaseManager dbManager = new
        StudentDatabaseManager();
192     private static Scanner scanner = new Scanner(System.in);
193
194     public static void main(String[] args) {
195         try {
196             System.out.println("=== CSPIT Student Management
                System ===");
197             System.out.println("Initializing...");
198
199             // Load existing data
200             try {
201                 dbManager.loadFromFile();
202             } catch (IOException | ClassNotFoundException e) {
203                 System.out.println("Warning: Could not load data
                    - " + e.getMessage());
204             }
205
206             // Main menu
207             while (true) {
208                 try {
209                     displayMenu();
210                     int choice = Integer.parseInt(scanner.
                        nextLine());
211
212                     switch (choice) {
213                         case 1:
214                             addStudent();
215                             break;
216                         case 2:
217                             viewAllStudents();
218                             break;
219                         case 3:
220                             searchStudent();
221                             break;
```

```
222         case 4:
223             viewTopStudents();
224             break;
225         case 5:
226             saveData();
227             break;
228         case 6:
229             System.out.println("Exiting system...
230                                 ");
231             return;
232         default:
233             System.out.println("Invalid option.
234                                 Please try again.");
235     }
236     } catch (NumberFormatException e) {
237         System.out.println("Please enter a valid
238                             number.");
239     } catch (Exception e) {
240         System.out.println("An error occurred: " + e.
241                             getMessage());
242     }
243 }
244
245 }
246
247 } catch (Exception e) {
248     System.err.println("Fatal error: " + e.getMessage());
249     e.printStackTrace();
250 }
251
252 private static void displayMenu() {
253     System.out.println("\n=== Main Menu ===");
254     System.out.println("1. Add Student");
255     System.out.println("2. View All Students");
256     System.out.println("3. Search Student by ID");
257     System.out.println("4. View Top Students");
258     System.out.println("5. Save Data to File");
259     System.out.println("6. Exit");
260     System.out.print("Enter your choice: ");
261 }
262
263 private static void addStudent() {
264     try {
265         System.out.println("\n=== Add New Student ===");
266         System.out.print("Student ID (e.g., CSPIT2025001): ");
267         ;
268         String studentId = scanner.nextLine();
269
270         System.out.print("Name: ");
271         String name = scanner.nextLine();
272
273         System.out.print("Department (CE/IT/CSE/AI): ");
```

```
268         String dept = scanner.nextLine();
269
270         System.out.print("Number of subjects: ");
271         int numSubjects = Integer.parseInt(scanner.nextLine()
272             );
273         int[] marks = new int[numSubjects];
274
275         for (int i = 0; i < numSubjects; i++) {
276             System.out.print("Marks for subject " + (i+1) + "
277                 : ");
278             marks[i] = Integer.parseInt(scanner.nextLine());
279         }
280
281         // Create and add student
282         Student student = new Student(studentId, name, dept,
283             marks);
284         dbManager.addStudent(student);
285
286     } catch (InvalidStudentDataException e) {
287         System.out.println("Validation Error: " + e.
288             getMessage());
289     } catch (StudentNotFoundException e) {
290         System.out.println("Database Error: " + e.getMessage
291             ());
292     } catch (NumberFormatException e) {
293         System.out.println("Invalid number format: " + e.
294             getMessage());
295     } catch (Exception e) {
296         System.out.println("Unexpected error adding student:
297             " + e.getMessage());
298     }
299 }
300
301 private static void viewAllStudents() {
302     System.out.println("\n=== All Students ===");
303     List<Student> students = dbManager.getAllStudents();
304
305     if (students.isEmpty()) {
306         System.out.println("No students registered.");
307         return;
308     }
309
310     for (Student s : students) {
311         System.out.println(s);
312     }
313
314     System.out.println("\nTotal Students: " + students.size()
315         );
316 }
317
318 private static void searchStudent() {
319     try {
```

```
311         System.out.println("\n=== Search Student ===");
312         System.out.print("Enter Student ID: ");
313         String studentId = scanner.nextLine();
314
315         Student student = dbManager.findStudent(studentId);
316         System.out.println("Student found:");
317         System.out.println(student);
318         System.out.println("Detailed Marks: " + Arrays.
            toString(student.getMarks()));
319
320     } catch (StudentNotFoundException e) {
321         System.out.println("Search Error: " + e.getMessage());
322         ;
323     }
324 }
325
326 private static void viewTopStudents() {
327     try {
328         System.out.println("\n=== Top Students ===");
329         System.out.print("How many top students to view? ");
330         int topCount = Integer.parseInt(scanner.nextLine());
331
332         List<Student> topStudents = dbManager.getTopStudents(
            topCount);
333
334         if (topStudents.isEmpty()) {
335             System.out.println("No students available.");
336             return;
337         }
338
339         System.out.println("Top " + topStudents.size() + "
            Students:");
340         for (Student s : topStudents) {
341             System.out.println(" " + s);
342         }
343     } catch (NumberFormatException e) {
344         System.out.println("Invalid number: " + e.getMessage());
345     } catch (IllegalArgumentException e) {
346         System.out.println("Error: " + e.getMessage());
347     }
348 }
349
350 private static void saveData() {
351     try {
352         dbManager.saveToFile();
353     } catch (IOException e) {
354         System.out.println("Error saving data: " + e.
            getMessage());
355     }
```

```
356     }  
357 }
```

Listing 9: CSPIT Student Management System with Exception Handling

8 Exception Handling Summary

8.1 Key Takeaways

- **Exception Hierarchy:** All exceptions derive from `Throwable`
- **Checked vs Unchecked:** Checked exceptions require handling; unchecked are optional
- **Try-Catch-Finally:** Basic structure for exception handling
- **Throw and Throws:** `Throw` to explicitly throw; `Throws` to declare exceptions
- **Custom Exceptions:** Create application-specific exceptions for better error handling
- **Best Practices:** Be specific, log errors, use `finally` for cleanup, don't swallow exceptions

8.2 CSPIT Implementation Checklist

1. Identify potential error conditions in your application
2. Use checked exceptions for recoverable errors
3. Use unchecked exceptions for programming errors
4. Create custom exceptions for domain-specific errors
5. Log all exceptions for debugging purposes
6. Provide meaningful error messages to users
7. Clean up resources properly using `finally` or `try-with-resources`

End of Notes